SIADS 696: Milestone II Team #01 Report

Uber Driving/Riding : Predicting Outcomes and Discovering Patterns

Ching-Yao Lin(yaulin@umich.edu)/ **Kha Nguyen**(minhkha@umich.edu)/ **Naiwen Duan**(nduan@umich.edu)

Introduction

Problem Statement

The goal of this project is to move beyond simple cancellation analysis of the Uber dataset and uncover deeper insights by combining it with external data sources. A key challenge is collecting and standardizing these datasets, since Uber's booking data only covers 2024. Our analysis will focus on predicting ride outcomes, driver ratings, booking fares and identifying behavior patterns using features such as weather condition, booking status, distance, and ratings,etc.

Impact

Accurate booking prediction and deeper pattern insights can improve dispatching, and service reliability. Better understanding of user behavior and travelling cost supports smarter operational decisions, improving both customer experience, platform efficiency as well as revenue.

Motivation

This project aims to uncover natural ride-booking patterns and fare prediction in India Uber data. By segmenting trips based on time, behavior, and weather, we can better understand customer behavior and operational trends.

These insights support fleet allocation, pricing strategies, and cancellation management, helping improve both efficiency and customer experience. From a technical perspective, this work explores how to effectively cluster mixed-type data using unsupervised learning methods.

Methods & contributions

- **Supervised Learning:** A comparison analysis was used to determine the best model family for each prediction problem we proposed. We were able to identify features that have an impact on booking completion probability, and train a booking fare prediction model that can be used to get an estimation of travelling cost in India.

- **Unsupervised Learning:** We applied K-Means and Agglomerative Clustering to detect hidden booking patterns in the Uber dataset, with t-SNE used for visualization. Unlike many studies that focus only on prediction, we expanded our supervised analysis by adding an unsupervised segmentation layer, making the patterns easier to interpret for business insights and to be aware of customer groups' common characteristics. .

Projects Findings

- **Supervised Learning:** We found that the pick up location and ride distance actually have a significant influence on booking completion status, and no feature has a strong effect (all features have relatively small coefficient) on customer booking rating or booking fare.
- **Unsupervised Learning:** Our unsupervised analysis uncovered distinct ride-booking patterns that reflect different user behaviors and weather conditions. Some clusters represented short, frequent trips with lower values, while others captured higher-value rides influenced by factors like operating hours and weather.

Related Work

- **Food Delivery Time Prediction study** — This study predicted delivery times in Indian cities using traffic, weather, and location data, employing models such as Random Forest and LightGBM. Our project differs in that it focuses on Uber rides, predicting outcomes such as completion, cancellations, and ratings, and utilizes clustering to identify patterns among customers and drivers.
- **Comparative Study of Uber and Lyft** —This paper uses machine learning models to analyze how Uber and Lyft's strategic choices (e.g., expansion and diversification) affected their growth and revenue. It focuses on business strategy forecasting and revenue simulation. Instead of strategic forecasting, we included weather related features and we also focused on ride-level patterns using unsupervised learning to uncover how temporal, behavioral, and weather factors shape booking behavior and segment ride types.
- **A Study and Analysis of Cabs by Ratings** — This paper examines customer ratings as a key service performance indicator. It identifies how driver behavior, trip distance, and service speed influence customer satisfaction. Such findings support our inclusion of rating features in clustering and anomaly detection, linking service quality with operational patterns.

Data Source(s)

This combination of datasets will be used for both Part A and Part B in our Milestone II project.

Main dataset: *Uber Ride Analytics 2024* ***(Data Scheme: Appendix A)***

This dataset provides a **comprehensive view of Uber ride-sharing activity** throughout the year 2024. It includes detailed information on booking behavior, vehicle usage, revenue generation, cancellations, and rider–driver interactions, making it ideal for both supervised and unsupervised modeling. The full dataset is available in CSV format which can be downloaded from Kaggle. The dataset records a total of **148,770** ride bookings across various vehicle types, capturing the full booking lifecycle — from ride request to completion or cancellation. It also reflects patterns in service performance, driver rating, customer behavior, and financial payment metrics.

Secondary datasets:

OpenRouteService API ***(Data Scheme: Appendix B)***

This dataset is used to obtain geographical coordinates (latitude and longitude) and full address corresponding to a specified location name from the main dataset. The return format of the API is JSON with the keys are location features which we can extract, we generated roughly **600 API responses** to get the address and coordinate of

booking location. DataCommon API was also used to generate a population density feature at state level, which only required a few API calls.

Historical Weather API ***(Data Scheme: Appendix C)***

Using the coordinates we got from Open Route Service for each booking, we generate the historical weather data (wind,temperature, etc.) for each booking location using this API, we only get the data for 2024 which is the date range of the main dataset. Roughly **500 API calls** were used to generate the dataset. The API is public and has a limit call per hour so this data collection step took quite a few hours to finish.

Data Preprocessing and Feature Engineering

The preprocessing stage of the project focused on enriching, cleaning, and structuring the data to make it suitable for both supervised and unsupervised learning. First, we merged the hourly weather data with the ride booking records based on address, date, and hour to capture environmental conditions such as temperature, humidity, and precipitation at the time of pickup. This allowed us to add contextual factors to the ride information without losing booking granularity.

Next, we addressed missing values in several key operational columns including waiting time, trip duration, booking value, ride distance, and both customer and driver ratings. For each of these variables, we created missing flags to preserve cancellation and data gap patterns as potential signals for clustering. We then imputed missing values using the mean for each vehicle type, ensuring that the imputed values reflected the behavior of similar rides rather than distorting the overall distribution. However, when we examined the average values across different vehicle types, we found that they were relatively similar. Because of this low variability between groups, we applied scaling to the entire dataset rather than scaling within each vehicle type. This ensured consistency between features and simplified preprocessing for both the supervised and unsupervised models.

Finally, we retained both the filled columns (with a "_fill" suffix) and their corresponding missing flags as separate features. This dual structure allowed the clustering algorithms to account for both the imputed values and the missingness itself.

We also include scaling, one-hot-encoding and correlation analysis before fitting the model; more details can be found in the supervised and unsupervised section. The list of full dataset features can be found in **Appendix D**.

Part A. Supervised Learning

Methods description

We approached booking-time prediction as a supervised learning problem and established a workflow to prevent data leakage. We started with a cleaned dataset, retaining only features available at booking time, such as booking hour, pickup location, vehicle type, and weather. Outcome signals were eliminated. We validated our approach with a chronological split mimicking a "*train on the past, predict the future*" method using grouped splits by Booking ID to avoid overlap. For feature processing, we applied median imputation and standardization for numeric features, while categorical features used most-frequent imputation and one-hot encoding, all within scikit-learn pipelines for consistency.

We evaluated three different model families for our prediction problems: *Logistic* (probabilistic model)//*Ridge Regression* (regularized linear regressor) chosen for interpretability and serve as a baseline, *Random Forest/Decision tree* (tree-based ensemble) for its ability to capture nonlinearity interactions and robustness to feature scale, and *k-Nearest Neighbours* (instance-based) as a non-parametric method to capture high dimensional structure in the data.

A targeted hyperparameter search was conducted to tune parameters for each model. We selected the best model based on validation F1 scores, refit it on the full training set, and reported the F1 score and ROC AUC on the held-out set for a fair deployment score that the team can build upon. We chose F1 score as a primary metric to balance precision and recall weights when evaluating. For the regression model, we used the RMSE metric instead, as it penalizes larger errors, and is more interpretable because it has the same unit as the target feature.

Supervised Evaluation

To get a clear and fair picture of how our models were performing, we didn't want to rely on just one test run. Instead, we used a 10-fold cross-validation approach for each of our prediction problems. This is a more robust way to assess how well the models generalize, as it tests them on multiple slices of the data.

Below, you'll find a summary table showing the average score each model achieved across the 10 folds, along with its standard deviation to indicate the consistency of the scores.

How to choose the right metrics?

The metrics we used were picked specifically for each task:

To determine whether a booking would be completed or cancelled, we used the F1-macro score. Our dataset has way more completed rides than cancelled ones, so this metric was perfect because it gives equal weight to both outcomes. This way, we could ensure the model wasn't simply achieving a high score by ignoring the less common "cancelled" category.

When predicting fares and ratings, we used the Root Mean Squared Error (RMSE). We chose RMSE because it effectively penalizes significant errors in our predictions. This is especially important for something like fare prediction, where being off by a large amount is more significant than being off by a small amount. Additionally, the error is expressed in the same units as the price, making it easier to understand.

What do we see from the result?

The 10-fold cross-validation results made it pretty clear which models were the best for each job:

Booking Completion: The Random Forest Classifier was the clear winner here. It hit a mean F1-macro score of 0.930 ± 0.004, easily beating out Logistic Regression and K-Neighbours. This indicates that its tree-based structure was effective in navigating the complexity of this problem.

Uber Fare Prediction: For predicting the price of a ride, Ridge Regression proved to be the most effective method. It had the lowest average error (RMSE of 1.577 ± 0.043), suggesting that a simple, regularized linear model was the best fit for our fare data.

Booking Rating Prediction: Ultimately, when attempting to predict customer ratings, the Decision Tree Regressor narrowly outperformed the other models, achieving the lowest RMSE of 0.345 ± 0.004.

What's interesting is that a different type of model won each contest. This really confirms that testing out a variety of methods was a good strategy, as there's no single "best" model for every type of problem.

Overall results reporting

Table 1: Metric comparison between model families

Prediction problem 1: Booking completion prediction (10-fold CV was used, chosen model is highlighted)			
Model	Logistic Regression	Random Forest Classifier	KNeighbors Classifier
F1-macro score with std	0.746211 ± 0.005649	0.930493 ± 0.004182	0.728861 ± 0.005054
Evaluation set score	0.748978	0.928174	0.747489
Prediction problem 2: Uber Booking fare prediction (10-fold CV was used, chosen model is highlighted)			
Model	Ridge Regression	Decision Tree Regressor	KNeighbors Regressor
RMSE score with std	1.577236 ± 0.043455	1.601062 ± 0.042169	1.653887 ± 0.042256
Evaluation set score	1.682957	1.702391	1.726743
Prediction problem 3: Booking rating prediction (10-fold CV was used, chosen model is highlighted)			
Model	Ridge Regression	Decision Tree Regressor	KNeighbors Regressor
RMSE score with std	0.346872 ± 0.004480	0.345353 ± 0.003888	0.354235 ± 0.004352
Evaluation set score	0.347756	0.346900	0.357205

In-Depth Evaluation

For the supervised learning part, we evaluate the performance of the models for three prediction problems: booking completion, uber fare prediction, and booking rating prediction. Detailed visualization and evaluation results can be referring to notebook (4.1-4.3 for lite dataset training, 4.4-4.6 for full dataset training)

Feature importance and Ablation Analysis

We used both the trained model coefficients and permutation importance test to evaluate which features are the most important for the model results.

Table 2: Model important features

Model	**Booking completion**	**Fare prediction**	**Booking rating prediction**

Most important features	Riding distance	All features in the model have relatively close coefficient so none is much more important than others	Pick up longitude, locality, booking hour
	Pick up information (latitude, longitude, region, locality, etc.)		Weather information at drop-off location (wind speed, temperature

Figure 1: Booking completion - Feature importance plot

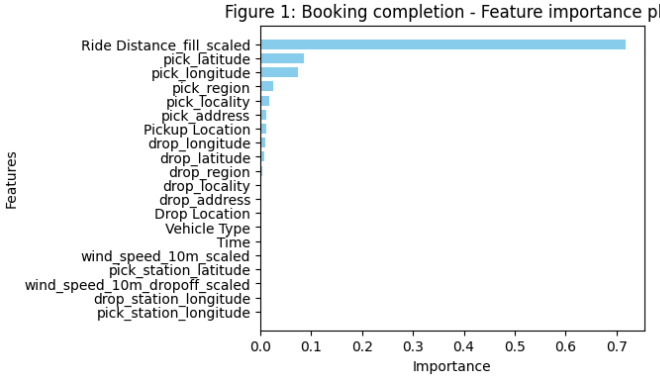
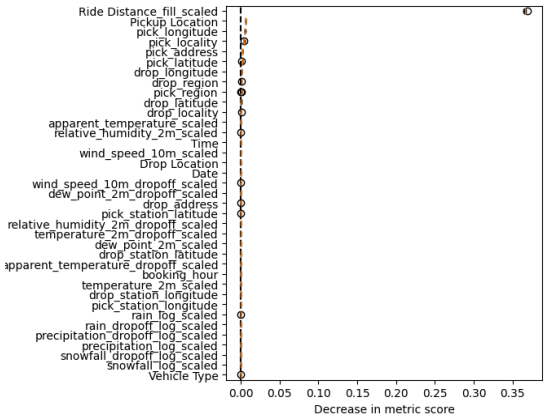


Figure 2: Booking completion - Permutation importance on train data

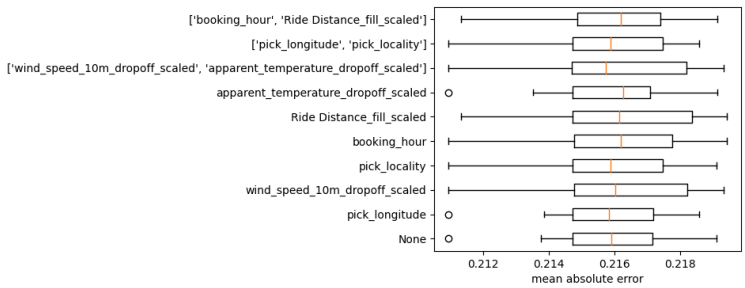


We also carry out different ablation tests on whether removing those important sets of features will affect the model performance. Surprisingly, for booking fare and booking rating prediction, removing those don't have a big impact on the model performance, this suggests that those features seem to have relatively medium/low effect on the prediction. On the other hand, removing **Ride distance** and pick-up location geographical features seem to have a big impact on the classification metric.

Table 3: Booking completion prediction ablation test result

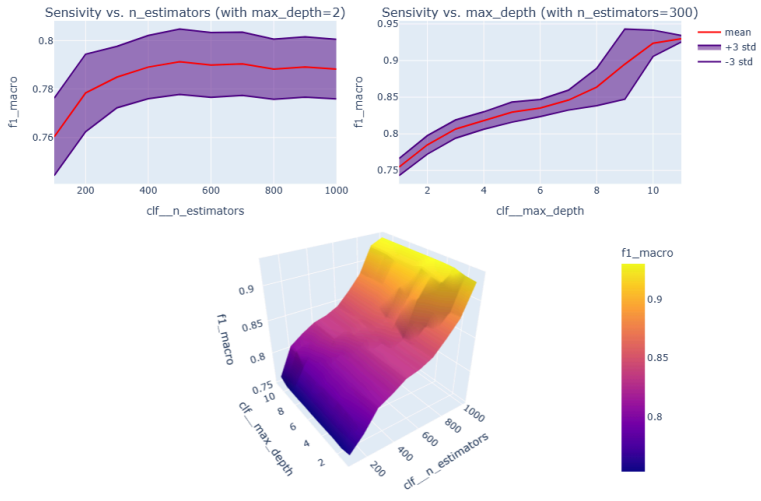
Ablated features	None	Ride distance	Pick up geographical features	Ride distance and pick up features
F1 macro score (using 10-fold CV)	0.929 ± 0.0014	0.71 ± 0.0039	0.92 ± 0.002	0.62 ± 0.0039

Figue 3: Booking rating - Ablation test



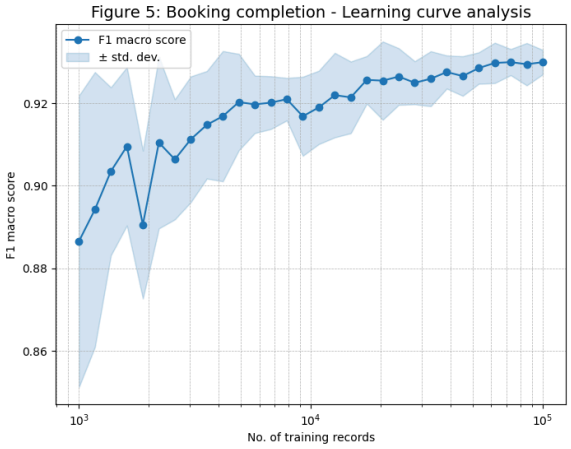
Sensitivity Analysis

Figure 4: Booking completion sensivity test



We carried out the sensitivity test to see how stable our model would be even with the twist of its hyperparameters. From the result of the booking completion sensitivity test. We can see that when checking against a test dataset, increasing the number of **n_estimators** in the **Random Forest** model can still further improve the model performance. While on the other hand, the model isn't very sensitive to the change in **max_depth** chosen. If training time isn't a big issue, we might consider increasing the number of **n_estimators** to get better metrics on the test set.

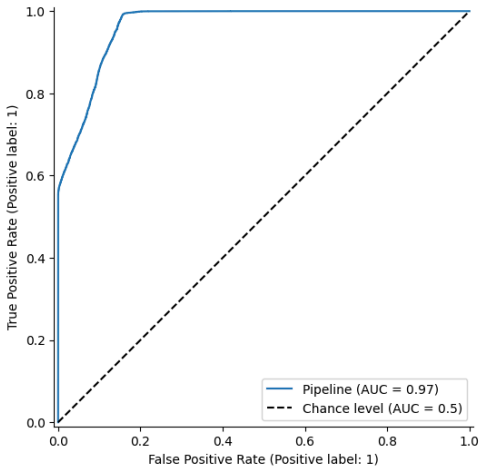
Trade-off analysis



We have conducted learning curve analysis on all three prediction problems from 1,000 records to 100,000 records. We found an interesting common insight, that while increasing the number of records improves the metrics, using more than 10,000 records for training **doesn't give a significant boost** to the performance metrics while taking much longer time to train the model. Given this insight, in future training circles and analyses, we can train only on a subset of the dataset without suffering too much on its performance.

Also, looking at the ROC curve of the booking completion problem, we can see that the area under the curve is very large, which indicates the model is performing very well and much better than tossing a coin. We are likely to be able to identify which booking will be completed with high accuracy, using this information, we can assign the booking information to suitable drivers and handle those that are likely to get cancelled accordingly.

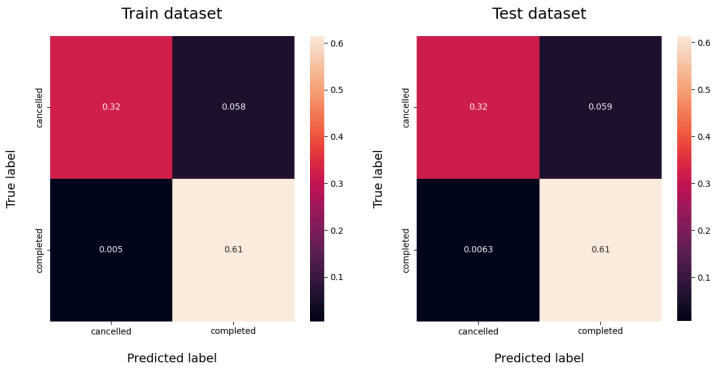
Figure 6: Booking completion - ROC curve



Failure analysis

We are going to look at the failure analysis for the booking completion prediction problem, this is a classification problem; and based on our dataset, we are expected to find some edge cases or misclassification due to model bias. We first look at the normalized confusion matrix of the model; we can see that the dataset is quite imbalanced (we have more completion records than cancellation). This might also contribute to the high False Positive prediction (completed booking) in the model.

Figure 7: Boking completion - Confusion matrix (normalized)



We are going to look at 3 records in the test dataset which are False Positive. Each represents different types of error.

First is an edge case in the test dataset, we get a failed prediction record where the booking **ride distance** is larger/smaller than 2 times standard deviation from the mean booking distance. For those edge cases, the majority label is completion (874 vs 47 records). After using the model to get the predictions, we found that all wrong cases are False Positive (indicating that very long or short ride distances are never cancelled). We can conclude this example belongs to **edge case error** where the model fails to generalize its prediction.

Figure 8: Record idx-11473 / Edge cases: Booking completion False positive

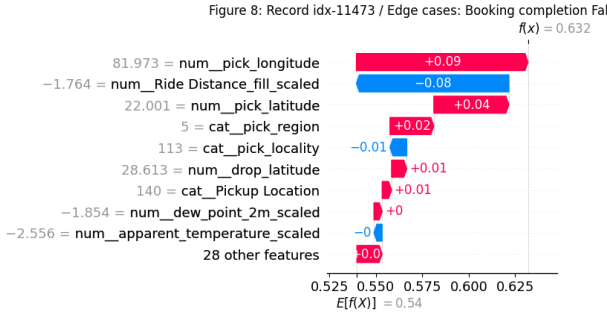
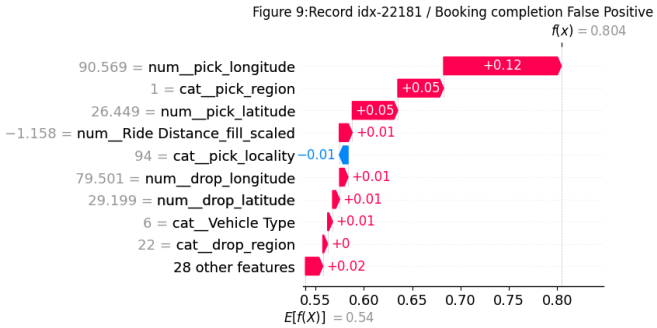


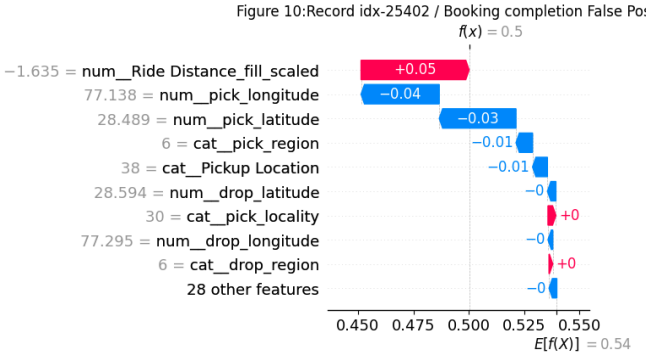
Figure 9:Record idx-22181 / Booking completion False Positive



Second is a False Negative case in the test dataset where the model is very sure about the probability of booking success. We can see that the location information features contribute greatly to the probability. This indicates that this is a **systematic error** where we can see a bias that booking from certain locations will contribute greatly to the booking completion.

Finally, this is an example record in the test dataset where the model makes wrong predictions due to the nature of its decision-making function. Any records that the model predicts just above or below 50% threshold are likely to get misclassified. Therefore, there is no consistent pattern, and this is a **random error** example.

Figure 10:Record idx-25402 / Booking completion False Positive



With the analysis above, we have some suggestions for future work to reduce those types of errors:

- While it's impossible to completely negative False positive or False negative error, we can twist the model decision function/threshold to reduce one while increasing the other.
- We can also try using data augmentation to increase the number of records for edge cases so that the model can generalize better when seeing unusual records.
- Because **ride distance** is such an important feature for booking completion prediction, perhaps we can consider collecting more features that will benefit the model by increasing its robustness while not depending too much on the **ride distance** booking feature.

Part B. Unsupervised Learning

Our main goal for the unsupervised part of the project was to find any natural, hidden groups or "segments" in our data, specifically looking for different customer behaviours or trip patterns. To achieve this, we examined two different types of clustering algorithms to determine if they'd yield similar results, thereby increasing our confidence in our findings.

Before we could cluster, we had to prep our data. Our workflow for this was pretty straightforward: we imputed any missing numbers using the median value of that column and filled missing categories with the most frequent one. Then, we converted all our categorical (text) features into numbers using an OrdinalEncoder. We chose this method because it's fast and turns our entire dataset into a simple, all-numeric format, which is exactly what these clustering algorithms need to do their job.

Here are the two methods we used:

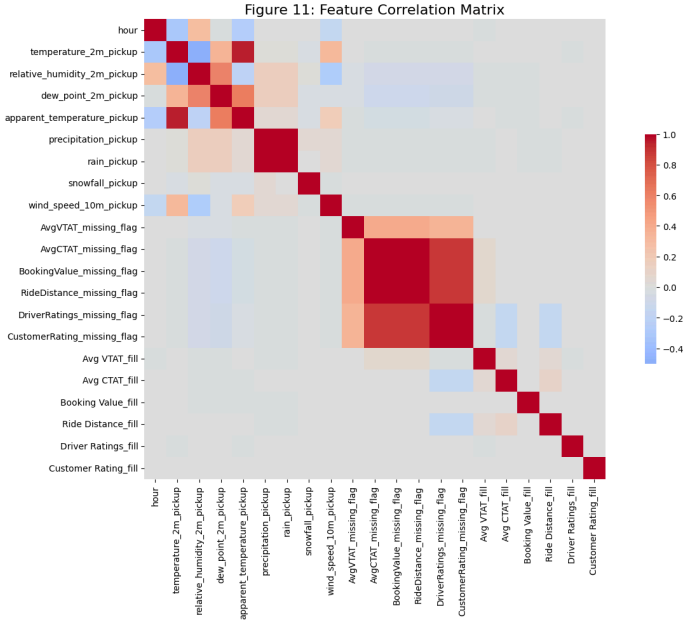
1. K-Means Clustering (Partitioning Method)

- **Why we chose it:** We started with K-Means because it's a classic, efficient, and really popular **partitioning** algorithm. It's great at taking a big dataset like ours and sorting it into a specific number of distinct, fairly balanced groups (or "clusters") by finding the center point for each group. We justified using it as our main method to find broad, clear-cut customer segments.
- **How we explored it:** The big question for K-Means is always "how many clusters (K) should we use?" Instead of just guessing, we explored this by testing a list of different K values (specifically K=2, 3, 4, 5, 8, and 10). To decide which K was best, we ran the algorithm for each and then measured the Silhouette Score. This score basically tells you how similar points are to their own cluster compared to other clusters. The K value that gave us the highest (best) Silhouette Score was the one we selected as our optimal number of groups.

2. Agglomerative Clustering (Hierarchical Method)

- **Why we chose it:** To double-check our work and get a different perspective, our team also brought in Agglomerative Clustering. This is a hierarchical method, which is a totally different approach. Instead of forcing the data into a set number of groups, it builds clusters from the bottom up, starting with every point as its own cluster and then merging the closest ones together, step-by-step, until you just have one big group. This was a perfect comparison method because it lets you see the "family tree" of the data.
- **How we explored it:** With this method, our exploration was more visual. We generated a dendrogram, which is that tree-like chart. It allowed us to see how the data naturally groups together at different levels. We used this to identify the most logical places to "cut" the tree to form clusters, and then we compared these results to what K-Means found. Seeing that both methods pointed to similar segments gave us a lot more confidence in our final analysis.

Figure 11: Feature Correlation Matrix



Data Preprocessing

We refined our feature set based on the correlation analysis (Figure 11) to reduce redundancy and improve model efficiency. Variables with strong collinearity—such as apparent_temperature_pickup, rain_pickup, and several missing flag indicators—were dropped in favor of single representatives like temperature_2m_pickup, precipitation_pickup, and AvgVTAT_missing_flag. Core ride performance variables (Booking Value_fill, Avg VTAT_fill, Avg CTAT_fill, and Ride Distance_fill) were retained to capture booking and operational patterns, while time (hour) and weather features were kept to provide contextual variability. This streamlined feature set preserves essential information while improving clustering performance and interpretability.

We applied one-hot encoding to *Vehicle Type* to represent each category as a binary feature, avoiding bias in distance-based algorithms like K-Means and Agglomerative Clustering. RobustScaler was used to handle outliers and reduce the impact of high-magnitude features. We performed correlation analysis to remove redundant variables and used a 30% stratified sample to maintain distribution while improving runtime.

We also tested FAMD and UMAP for dimensionality reduction, but they produced unstable cluster separations due to data complexity and size. One-hot encoding with robust scaling gave the most stable and interpretable results.

Unsupervised Model Evaluation

This evaluation compares clustering performance across k values using multiple metrics. The Silhouette Score peaks at k = 2 but remains strong at k = 4–5. The Davies–Bouldin Index reaches its minimum at k = 4, indicating well-defined clusters. The Calinski–Harabasz Index increases with k, and the Elbow Method shows diminishing returns beyond k = 4–5. Overall, k = 4 provides the best balance of separation and compactness, with k = 5 as a reasonable alternative for finer granularity.

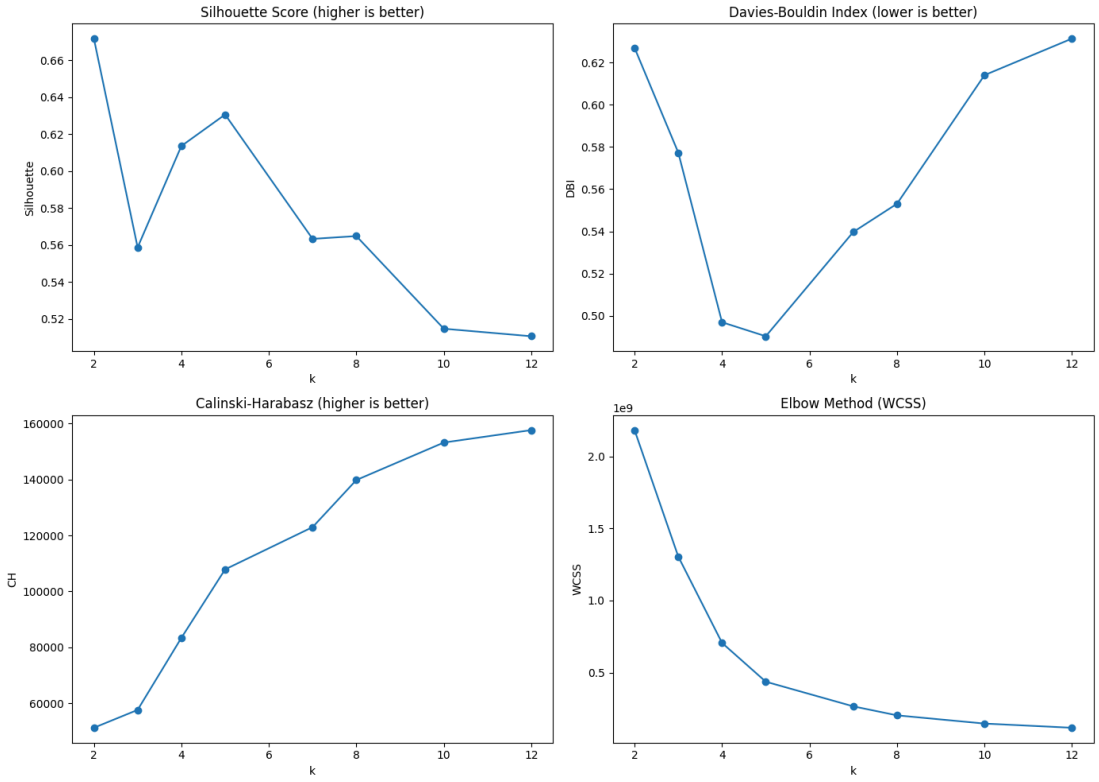


Figure 12: Evaluation of KMeans Clustering

To complement these quantitative metrics, the dendrogram analysis (Figure 13) from hierarchical clustering provides a visual perspective on natural cluster formation. By examining the large vertical jumps in the dendrogram, a cut height at 20,000 was identified, which corresponds closely to the k = 4–5 range suggested by the other metrics. This alignment between hierarchical structure and KMeans metric-based evaluation further validates the choice of cluster number and indicates that the data has a clear, stable segmentation pattern at this level.

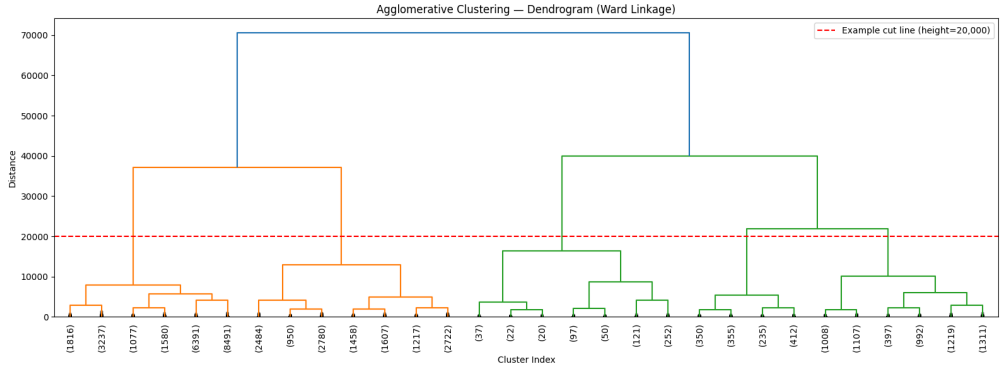


Figure 13: Hierarchical dendrogram

We had additional graphs to test on the number of clusters to pick. T-SNE was used after fitting the models which were not used for training but provided meaningful 2D projections to evaluate cluster separation and distribution. From Figure 14, comparing k = 4 and k = 5, both methods produce well-separated clusters with clear boundaries. At k = 4, the clusters are larger and more compact, providing a simpler segmentation. At k = 5, the data is divided into finer segments — one of the larger clusters splits into two, revealing a more granular structure.Since the computing time does not differ much from 4 clusters to 5. We choose KMeans with 5 clusters since we aim to include more patterns but less computing power in the analysis.

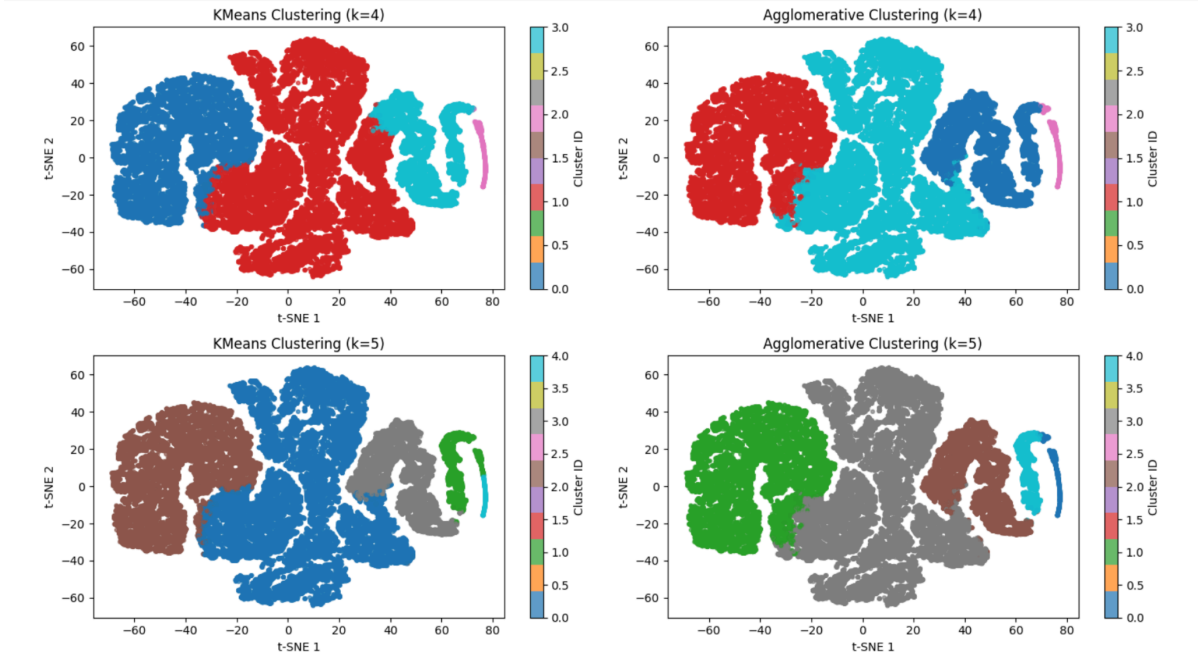


Figure 14: KMeans and Agglomerative Clustering Comparison

The t-SNE visualization of the clustering results with k = 5 (Figure 15) reveals five distinct ride segments in the Uber dataset. We use bubble size to represent booking value, enabling us to assess both segment distribution and economic contribution. Clusters 0 and 2 account for the largest share of rides, corresponding to short, frequent, low-value trips.

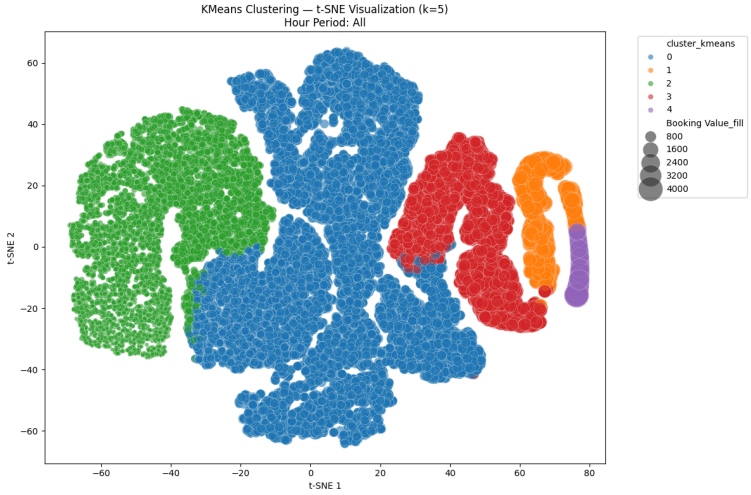


Figure 15: Clustering & Booking Value

In order to see what makes cluster 4 stand out, we conduct a feature deviation analysis that reveals clear behavioral differences between clusters. Cluster 4 stands out with a 404.8% higher booking value than the overall mean, indicating it captures heavier rain perceptions, high-value rides. In contrast, Clusters 0 show low booking values and a dominance of Auto and Go Mini trips, aligning with short, low-fare rides. We also observed a relatively high proportion of missing average VTAT flags in Cluster 0, which suggests that many of these trips were canceled before being assigned to a driver. Aside from this feature, Cluster 0 exhibits fairly low deviation across most other features, indicating that this cluster largely represents low-value trips without strong deviation.

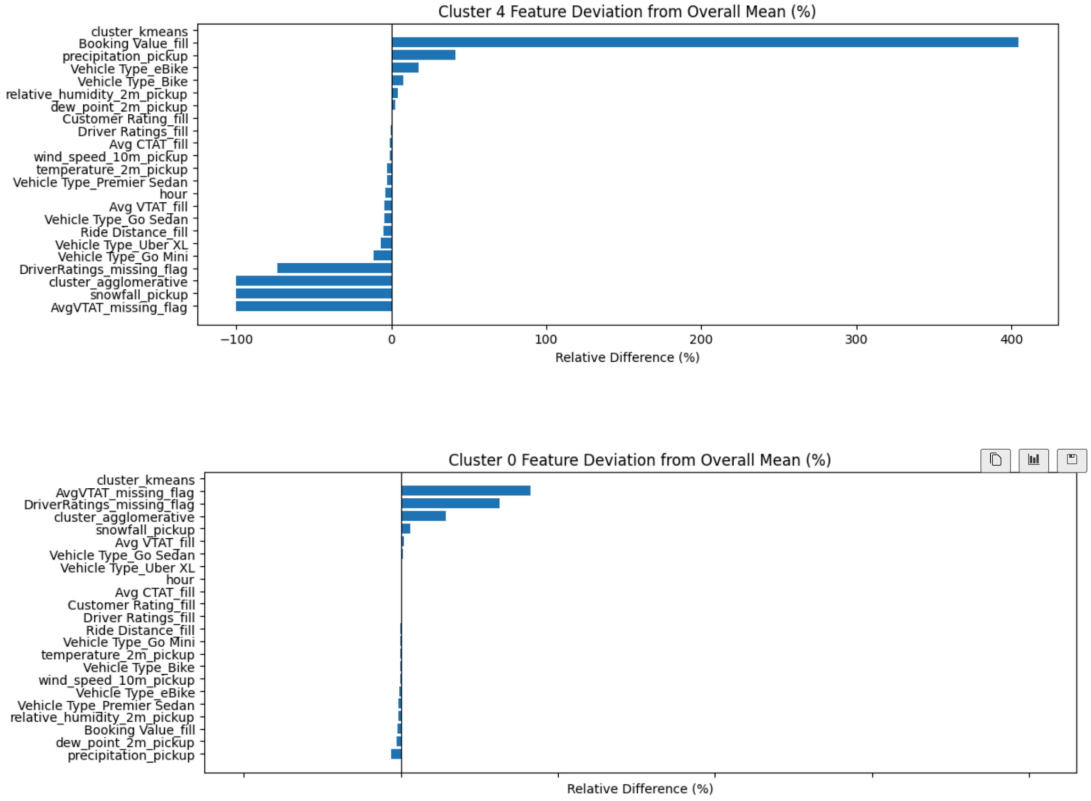


Figure 16: Feature Deviation

Unsupervised Learning Additional Findings

- Parameter sensitivity — Small changes in k values affect cluster granularity, particularly between k = 4 and k = 5, where one large cluster splits into two finer segments.

- Dimensionality reduction — UMAP and FAMD did not provide stable separations due to mixed-type data and scaling sensitivity. Using Gower distance or more efficient dimensionality reduction methods could improve structure detection in future iterations.

- Additional feature refinement — Current segmentation captures broad behavioral patterns. More granular features (e.g., geospatial zones) could improve interpretability and business relevance.

Discussion

What did you learn from doing Part A?

From Part A, we learned that having a realistic setup, focused solely on booking-time features and using a grouped time-based split, is more critical than selecting complex models. This approach led to believable and stable scores. We were surprised to see how an F1 score of approximately 0.95 dropped to around 0.85 when we removed post-outcome signals and eliminated the overlap between Booking and Customer IDs. This confirmed that the earlier result was due to leakage rather than true generalization.

One of the main challenges we faced were leakage and ID overlap, along with a few issues related to duplicate columns and paths. We addressed these problems by adhering to a strict policy of utilizing only booking-time features, conducting ablation tests to justify exclusions, implementing a grouped time split with a gap, and establishing pipelines for imputation and encoding.

Another challenge we faced were the computation power required in order to run the training process, especially the Random Forest model. In order to tackle this problem, we decided first to train a lite version of the dataset. We use the top 20 features and only 20% records to train the model, then conduct a learning curve analysis to see if more records are required. After finishing the training and evaluation of the lite dataset, we still had time so we tried to set up Great Lake HPC and use the extra computational power to train the full dataset, which we finished on time.

What surprised us was that for customer booking rating and booking fare, we couldn't find any strong features that have an impact on the model metric (when we remove those) even though the coefficient of some features are higher than others.. It could be that the model choice wasn’t good, there might be some sort of leakage, or the features we have weren’t suitable for those problems.

With more time, we would incorporate rolling time cross-validation and probability calibration, engineer safer features (such as geohashes, seasonality, and weather interactions), explore gradient boosting with systematic tuning, and perform threshold tuning and drift monitoring for deployment.

What did you learn from doing Part B?

Through Part B, I learned how crucial data preprocessing is for effective unsupervised learning. Early attempts using FAMD and UMAP resulted in weak, overlapping clusters because of skewed numerical features and high dimensionality from one-hot encoding. After applying Robust scaling, the feature space became more balanced, allowing K-Means and Agglomerative Clustering to produce more stable and interpretable clusters.

I also learned that feature selection directly impacts cluster quality. Temporal and operational variables—such as hour, vehicle type, and weather condition—played a key role in separating meaningful patterns. Overall, this part of the project showed how the right combination of preprocessing, scaling, and tuning can uncover valuable structure in unlabeled data.

With more time, I would explore Gower distance with K-Medoids to better handle mixed data types.. I could also incorporate domain-specific rules (e.g.,location, route, peak vs. off-peak periods) to refine segmentation and integrate supervised learning afterward to predict cluster membership for future rides. Finally, more computing power would allow training on the full dataset and tuning parameters more extensively.

Ethical Considerations

Part A: For each of the models we build to solve the problems we discuss. Each of them might have ethical issues when we want to apply them to production. Being able to predict whether a booking is going to be successful or not might create discrimination, users or drivers that often cancel or classify as high-cancellation chance by the model might be penalized and take a longer queue in the app or not prioritize for some services. In order to avoid this, we believe the booking completion prediction should be used as part of a traffic coordination tool where it would try to pair customers and drivers from different locations that give a better chance of completion. Another issue could be the customer booking rating prediction outputs, there might be a chance that the ratings are subjective and reflect biases (racial, gender, etc.), so if the model is used to evaluate drivers' attitude and performance, it wouldn't be fair for them. A thorough bias and fairness analysis would need to be done in order to be sure about the validity of the model in supporting the drivers' evaluation.

Part B: Our dataset includes sensitive information such as booking IDs, customer IDs, pickup and drop-off locations, trip distances, and ratings, there are significant ethical considerations around privacy and responsible data use. These details can reveal personal movement patterns(pickup and dropoff location), payment method, habitual travel behavior, and potentially identifiable locations like home or work. If unsupervised clustering results were applied without caution, they could expose or reinforce inequalities, enable location-based profiling, or lead to unfair business decisions. To address these concerns, we minimized the use of direct identifiers, applied appropriate feature engineering to reduce re-identification risks, and carefully interpreted clusters to avoid discriminatory or harmful conclusions. Also, because the dataset includes IDs and location data, secure storage, access control, and proper anonymization are critical. Poor data handling can lead to leaks or unauthorized use.

Statement of Work

Kha Nguyen	**Ching-Yao Lin**	**Naiwen Duan**
Data collection Data modeling Supervised learning advanced analyses and visualization Report Writing Repository maintenance	Data Modelling Supervised Learning Unsupervised Learning Report Writing	Data Preprocessing, Clustering Analysis, Unsupervised Model Visualization, Report Writing

References

- Laddha, Y. (2024). Uber Data Analytics Dashboard. Kaggle. Retrieved October 15, 2025, from https://www.kaggle.com/datasets/yashdevladdha/uber-ride-analytics-dashboard/data
- HeiGIT gGmbH. (n.d.). OpenRouteService API. Retrieved October 15, 2025, from https://api.openrouteservice.org/
- Open-Meteo. (n.d.). Weather forecast API. Retrieved October 15, 2025, from https://open-meteo.com/en/docs
- Sciki-learn documentations on evaluation methods and metrics, from https://scikit-learn.org/stable/user_guide.html

Appendix A — 2024 Uber Data Schema

Column Name	Description
Date	Date on which the ride was booked
Time	Booking time
Booking ID	Unique ride booking identifier
Booking Status	Final booking outcome (Completed, Cancelled by Customer, Cancelled by Driver, etc.)
Customer ID	Unique customer identifier
Vehicle Type	Type of vehicle requested (Go Mini, Go Sedan, Auto, eBike/Bike, UberXL, Premier Sedan)
Pickup Location	Ride starting point
Drop Location	Ride destination
Avg VTAT	Average time for driver to arrive at pickup (minutes)
Avg CTAT	Average trip duration (minutes)
Cancelled Rides by Customer	Indicates if the cancellation was customer-initiated
Reason for cancelling by Customer	Stated reason for customer cancellation
Cancelled Rides by Driver	Indicates if the cancellation was driver-initiated
Driver Cancellation Reason	Reason for driver cancellation
Incomplete Rides	Flag for rides that started but were not completed
Incomplete Rides Reason	Reason for incomplete rides
Booking Value	Total fare charged for the ride
Ride Distance	Total distance of the trip (in kilometers)
Driver Ratings	Rating given to the driver (1–5)
Customer Rating	Rating given by the customer (1–5)
Payment Method	Payment mode (UPI, Cash, Credit Card, Uber Wallet, Debit Card)

Appendix B — OpenRoad Service API Schema

Column Name	Description
address	places with a street address
region	states and provinces
county	official governmental area; usually bigger than a locality, almost always smaller than a region
locality	towns, hamlets, cities

Appendix C — Weather API Schema

Column Name	Data Type	Description
latitude	Float	WGS-84 latitude of the weather grid cell.
longitude	Float	WGS-84 longitude of the weather grid cell.
elevation	Float	Elevation of the cell (if provided).
timezone	String	Time zone of the returned weather data (e.g., “GMT+0”).
time	Timestamp	ISO-8601 timestamp (hourly) of the measurement.
temperature_2m	Float (°C)	Air temperature 2 meters above ground.
relative_humidity_2m	Float (%)	Relative humidity at 2 m above ground.
dew_point_2m	Float (°C)	Dew point temperature at 2 m above ground.
precipitation	Float (mm)	Sum of rain + snow in the previous hour.
rain	Float (mm)	Liquid precipitation only in the previous hour.
snowfall	Float (cm)	Snowfall amount in the previous hour.
wind_speed_10m	Float (km/h)	Wind speed at 10 m above ground.
wind_direction_10m	Float (°)	Wind direction at 10 m above ground.
weather_code	Integer	WMO weather condition code.

Appendix D — Processed Dataset final feature names

Date
Time
datetime
hour
Payment Method
Booking ID
Booking Status
Customer ID
Vehicle Type
Pickup Location
Drop Location
Avg VTAT_fill
Avg CTAT_fill
Booking Value_fill
Ride Distance_fill
Driver Ratings_fill
Customer Rating_fill
temperature_2m_pickup_scaled
relative_humidity_2m_pickup_scaled
dew_point_2m_pickup_scaled
apparent_temperature_pickup_scaled
wind_speed_10m_pickup_scaled
temperature_2m_drop_scaled
relative_humidity_2m_drop_scaled
dew_point_2m_drop_scaled
apparent_temperature_drop_scaled
wind_speed_10m_drop_scaled
precipitation_pickup_log
rain_pickup_log
snowfall_pickup_log
precipitation_drop_log
rain_drop_log
snowfall_drop_log
precipitation_pickup_log_scaled
rain_pickup_log_scaled
snowfall_pickup_log_scaled
precipitation_drop_log_scaled
rain_drop_log_scaled
snowfall_drop_log_scaled

Appendix E — Notebook Catalog

Notebook Category	Notebook Description	Notebook Name
Data collection & preprocessing	Collecting all raw data from secondary dataset (OpenRoute service and Historical weather Weather API)	1. Data collection.ipynb
	Data Merging	2.1. Data Preprocessing - Scaling.ipynb
	Data Cleaning and Filling N/A	2.2. Data Preprocessing - Demographic merging.ipynb
	Data Normalization	2.3. Data Preprocessing - Merging.ipynb
Supervised Learning	This notebook runs the complete supervised learning workflow for 3 prediction problems, including Booking completion (classification), fare prediction (regression) and rating prediction (regression). It trains and evaluates diverse model families using 10-fold cross validation to compare them. It also includes a "LITE" mode for rapid prototyping on a feature-selected dataset.	3. Supervised modeling.ipynb
	Supervised learning advance analysis & Failure analysis	Note book 4.1-4.3 are trained on lite dataset
	Supervised learning advance analysis & Failure analysis	Notebook 4.2-4.6 are train on full dataset
Unsupervised Learning	This notebook applies unsupervised methods to find patterns in the data. It includes correlation analysis, dimensionality reduction with parameter tuning, and runs clustering algorithms. The final step is the exploration and interpretation of the resulting customer segments.	5. Unsupervised modeling.ipynb
	Appy Correlation analysis	5.1. Unsupervised Model Correlation.ipynb
	Apply Scaled dimensionality reduction on mixed type data and parameter tuning	5.2. Unsupervised Model Evaluation & Sensitivity.ipynb